

ADR-DDD-001: Domain model and bounded contexts

החלטת ארכיטקטורה (ADR)

נוצר על ידי: אלכס • 2026-09-25

תוכן עניינים

[Consequences .4](#)[Alternatives considered .3](#)[Decision .2](#)[Context .1](#)

Context 1

PRD-01 (approved 2026-09-25) defines a resource-capacity ordering system with a hard correctness requirement (no oversold oven/work minutes, enforced at the DB level), a secondary write path that creates orders from a different flow (custom cakes), and several supporting concerns (catalog, delivery pricing, identity, notifications) that Yuval must be able to manage without a `domain-map.md`:developer. Full detail

Decision 2

Eight bounded contexts, one Next.js monolith, one Postgres database. No service split; the boundary is `lib/server/{context}/` folders plus the rule that only the owning context's module ever writes its own tables directly. Capacity is the core, invariant-bearing context; Ordering and CustomCake are both Conformists to it and must call its `reserve()` / `release()` functions

rather than reimplementing "is there room" (Rule 19: one shared definition of
.(a business quantity, never two

`CustomCakeRequest.approve()` The one deliberately-named cross-context write is
creating an `Order` : it happens inside a single DB transaction, not as an
async event, specifically so it can never race a standard-product checkout
for the same day's capacity. This is called out because it is the one place
in the system where "which context owns this write" could plausibly be
gotten wrong, and it is the highest-consequence place to get it wrong (real
.(money, real oven

Alternatives considered 3

No DDD structure, flat `lib/` folder. Rejected: PRD-01 already names .1
eight distinct concerns with different lifecycles (loyalty is Phase 2
capacity is correctness-critical, delivery has zero external
dependencies) and a flat folder would let a checkout handler reach
directly into the capacity table, which is exactly the bypass Rule 19
exists to prevent

Microservices per context. Rejected: single kitchen, single admin .1
single small team, no independent scaling need, and a service boundary
would turn the one transaction that must stay atomic (CustomCake approve
Capacity reserve + Order create) into a distributed transaction for no +
benefit. Wrong shape for this project's size

Consequences 4

jordan/dana build inside `lib/server/{domain}/` , never reach across into •
another domain's folder to write its tables

dba names tables per context (e.g., `capacity_day_ledger` , not a generic •

.so a table name alone tells you its owning context ,(`ledger`

amit-integration checks cross-context boundary violations as part of Rule •

.review 2

If Kivun takes a second capacity-scheduling client, this map is the •

reusable shape to start from (per engagement.md's Rule 16 note on

.(extending rather than forking domain knowledge

מאושר

מאושר